

实验二：原生应用开发实验报告

一、实验基本信息

项目	内容
实验名称	原生应用开发
实验类型	验证型
实验学时	4学时
开发平台	Android (Kotlin + Jetpack Compose)
项目名称	云端智能畜牧养殖管理系统

二、实验目的

- 掌握Android Kotlin登录页面开发
- 理解Jetpack Compose现代UI框架的使用
- 学会Activity/页面间数据传递
- 掌握MVVM架构设计模式
- 完成功能模块的完整实现

三、实验环境

项目	配置
操作系统	Windows 11
开发工具	Android Studio
编程语言	Kotlin
UI框架	Jetpack Compose
最低SDK	API 24 (Android 7.0)
目标SDK	API 36
构建工具	Gradle 9.3.1

四、实验内容

4.1 团队分工

根据项目需求，团队成员分工如下：

姓名	技术岗位	技术栈	主要功能
张玉满	Android开发	Kotlin+Android SDK	图表绘制、指纹辅助登录、视频监控、AI疾病诊断、离线模式、电池优化
赫宇豪	UniApp开发	Vue.js+CSS	组件应用、数据同步、云端打包、AI饲料配比优化、响应式布局、数据可视化增强
彭绍洋	HarmonyOS开发	ArkTS+HarmonyOS SDK	分布式设备管理、IoT数据采集、AI畜群健康分析、分布式存储、边缘计算
张爽	小程序开发	JS+WeChat MiniProgram	自定义组件、图表绘制、异常弹窗、AI预警系统、小程序生态集成、分享机制
李世程	MAUI开发	C#+.NET+MAUI	数据报表生成、图表展示、数据分析、AI生长预测模型、企业级安全、报表定制化

4.2 项目概述

本项目为"云端智能畜牧养殖管理系统"，主要功能包括：

- 用户登录验证
- 牧场数据仪表盘
- 牲畜信息管理
- 饲料库存管理

4.3 项目架构

采用MVVM架构设计，项目结构如下：

app/src/main/java/com/example/myapplication1/	
├─ data/	# 数据层
├─ Livestock.kt	# 牲畜数据模型
├─ Feed.kt	# 饲料数据模型
└─ FarmRepository.kt	# 数据仓库
├─ login/	# 登录模块
├─ LoginScreen.kt	# 登录界面
└─ LoginViewModel.kt	# 登录视图模型
├─ ui/	# UI界面层
├─ home/	# 主页导航
├─ dashboard/	# 仪表盘
└─ livestock/	# 牲畜管理

五、功能实现

5.1 登录功能实现

5.1.1 登录界面设计

登录界面采用Jetpack Compose实现，包含：

- 系统标题显示
- 用户名输入框
- 密码输入框（密码隐藏）
- 登录按钮
- 错误提示信息

【登录界面】

云端智能畜牧养殖管理系统

请输入账号

请输入密码

登录



5.1.2 输入验证逻辑

登录验证规则：

- 用户名和密码不能为空
- 正确账号：admin
- 正确密码：123456

【输入验证提示】

云端智能畜牧养殖管理系统

请输入账号

请输入密码

登录

账号和密码不能为空



5.1.3 登录成功跳转

登录成功后，系统通过Navigation组件跳转到主页，并传递用户名参数。

【登录成功跳转】

牧场管理中心

欢迎回来
admin

今天是 2026年03月28日

数据概览

牲畜总数

5

头/只

饲料种类

3

种

饲料库存

1700

公斤

牲畜分布



牛

2 头/只



羊

2 头/只



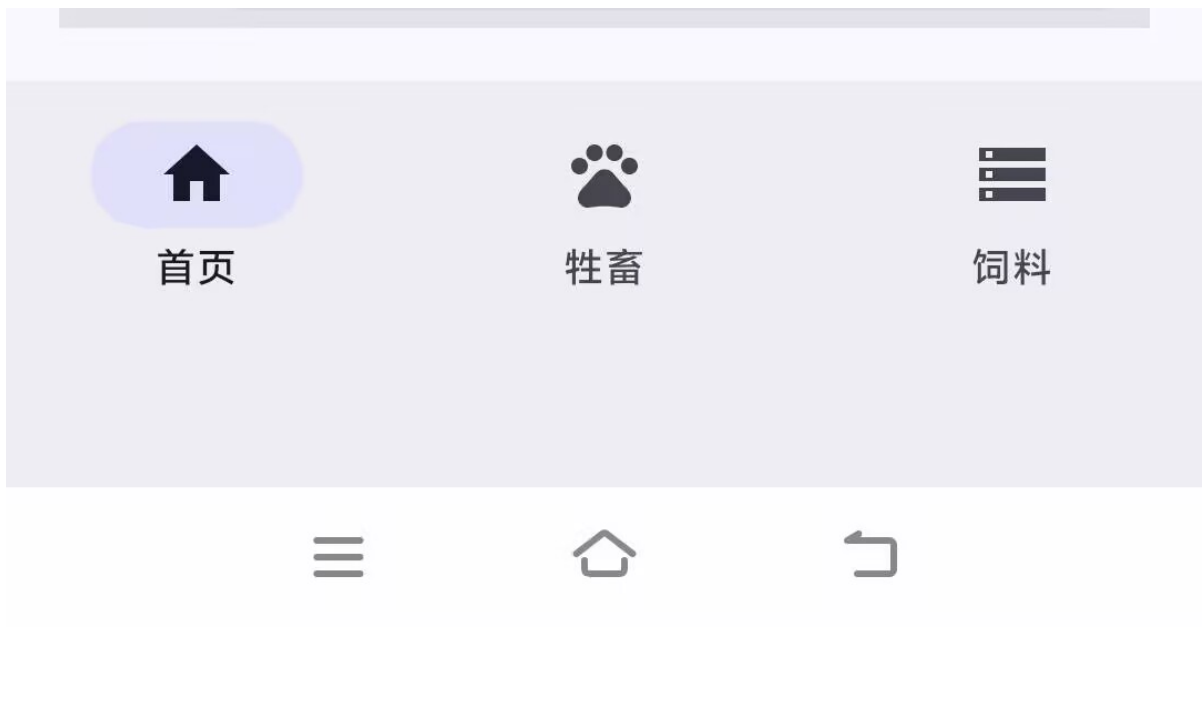
5.2 主页导航框架

5.2.1 底部导航栏设计

主页采用Scaffold布局，包含：

- 顶部标题栏：显示"牧场管理中心"
- 底部导航栏：三个Tab（首页、牲畜、饲料）
- 内容区域：根据选中Tab显示不同页面

【主页底部导航】



5.3 仪表盘功能

5.3.1 欢迎卡片

显示欢迎信息和当前日期，个性化展示登录用户名。

【欢迎卡片】

牧场管理中心

欢迎回来
admin

今天是 2026年03月28日

数据概览

牲畜总数

5

头/只

饲料种类

3

种

饲料库存

1700

公斤

牲畜分布



牛

2 头/只



羊

2 头/只





5.3.2 数据概览

展示牧场核心数据统计：

- 牲畜总数
- 饲料种类
- 饲料库存

5.4 牲畜管理功能

5.4.1 牲畜列表

展示所有牲畜信息，包括：

- 牲畜名称
- 种类和品种
- 年龄和体重
- 编辑和删除按钮

【牲畜列表】

牧场管理中心

大黄



牛 · 黄牛

年龄: 3岁 体重:
450.0kg



小白



羊 · 山羊

年龄: 2岁 体重: 35.0kg



花花



猪 · 土猪

年龄: 1岁 体重:
120.0kg





5.5 饲料管理功能

5.5.1 饲料列表

展示所有饲料信息，包括：

- 饲料名称
- 类型和供应商
- 库存数量
- 过期日期
- 即将过期提醒（7天内过期显示警告）

【饲料列表】

牧场管理中心

玉米饲料

精饲料 · 饲料供应商



A

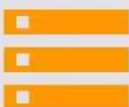


库存: 500.0公斤

过期: 2026-04-27

青草 即将过期

粗饲料 · 本地采购



库存: 1000.0公斤

过期: 2026-04-04

豆粕

精饲料 · 饲料供应商





六、测试验证

6.1 测试用例

测试项	测试内容	预期结果	实际结果
登录-空用户名	不输入用户名点击登录	提示"账号和密码不能为空"	通过
登录-空密码	不输入密码点击登录	提示"账号和密码不能为空"	通过
登录-错误凭证	输入错误账号密码	提示"账号或密码错误"	通过
登录-正确凭证	admin/123456	跳转到主页	通过
导航-Tab切换	点击底部导航栏	正确切换页面	通过
牲畜-添加	添加新牲畜	列表显示新数据	通过
牲畜-编辑	编辑牲畜信息	数据更新成功	通过
牲畜-删除	删除牲畜	从列表移除	通过
饲料-添加	添加新饲料	列表显示新数据	通过
饲料-过期提醒	查看过期提醒	显示警告标签	通过

七、技术要点总结

7.1 使用的技术栈

技术	用途
Kotlin	开发语言
Jetpack Compose	UI框架
Material Design 3	UI组件库

技术	用途
Navigation Compose	页面导航
ViewModel	状态管理
StateFlow	响应式数据流

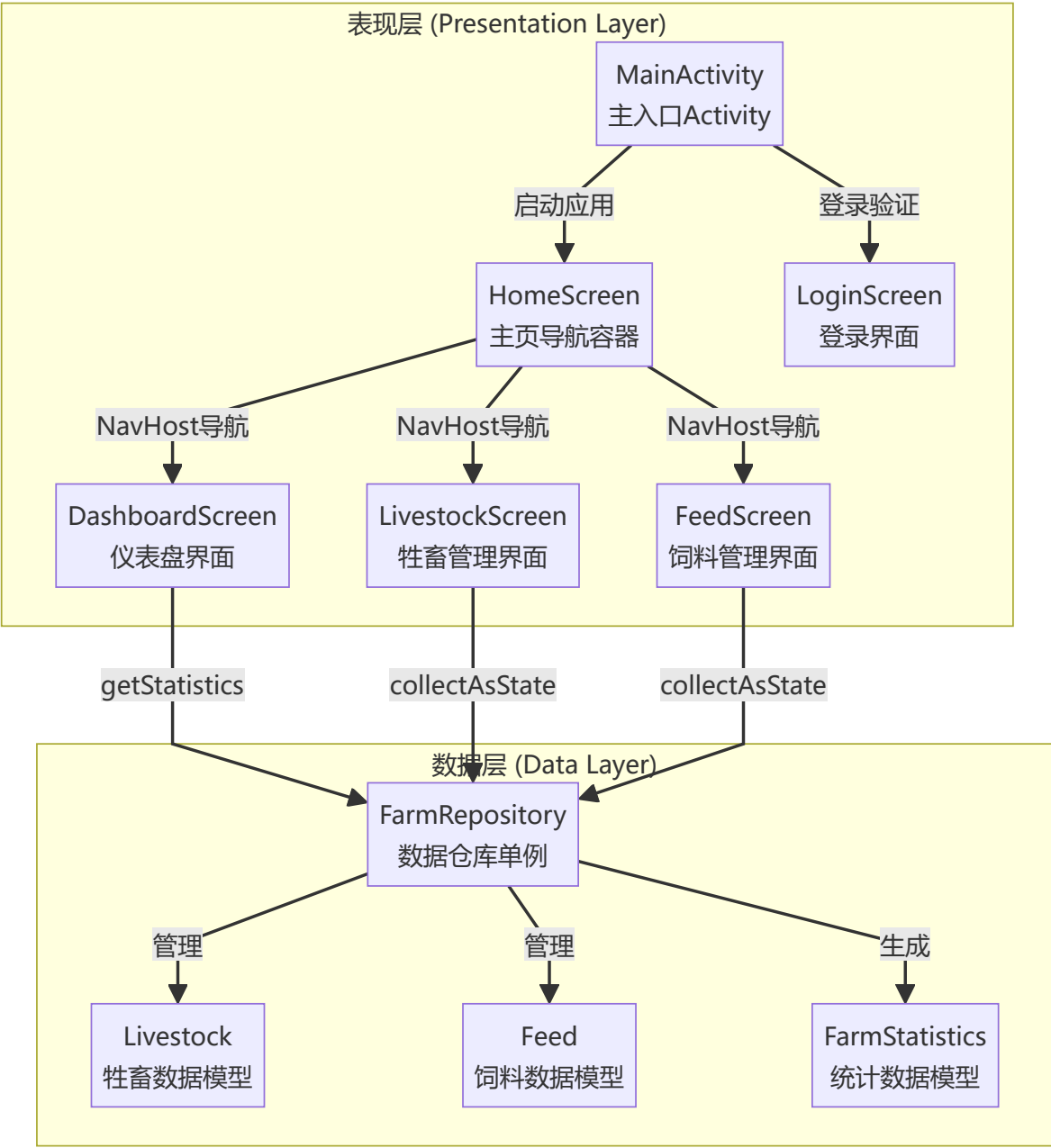
7.2 核心实现要点

- MVVM架构**: ViewModel管理UI状态, Repository管理数据
- 响应式UI**: 使用StateFlow实现数据变化自动更新UI
- 声明式UI**: Jetpack Compose声明式构建界面
- 单Activity架构**: 使用Navigation管理页面导航

八、系统架构设计

8.1 系统框架图

系统采用MVVM架构模式，分为表现层和数据层两层结构：

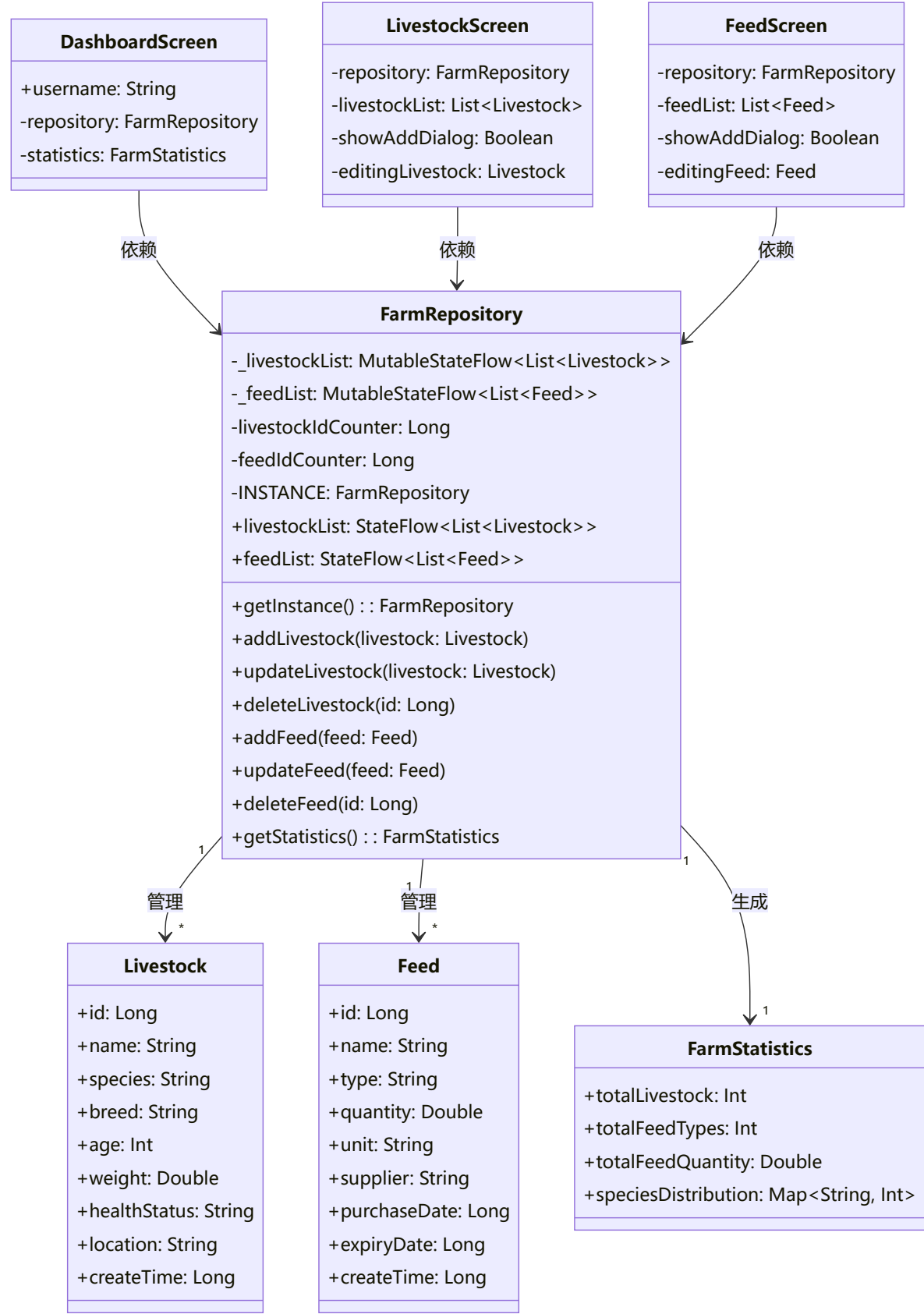


框架说明：

层级	职责	主要组件
表现层	UI展示与用户交互	MainActivity、各Screen组件、LoginViewModel
数据层	数据存储与管理	FarmRepository、数据模型

本系统核心功能模块（仪表盘、牲畜管理、饲料管理）直接使用 FarmRepository 进行数据操作，通过 StateFlow 实现响应式数据更新。

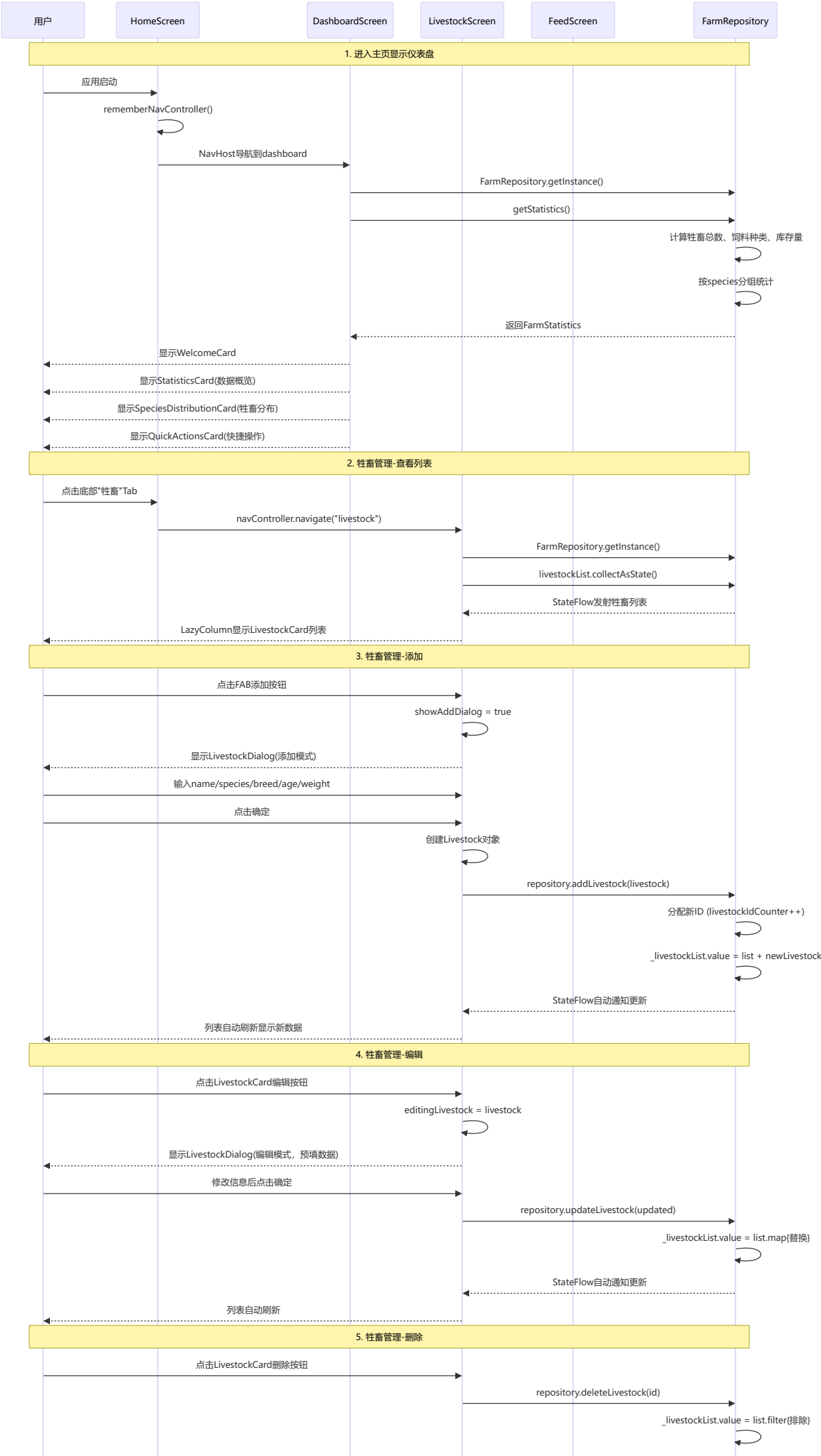
8.2 核心类图

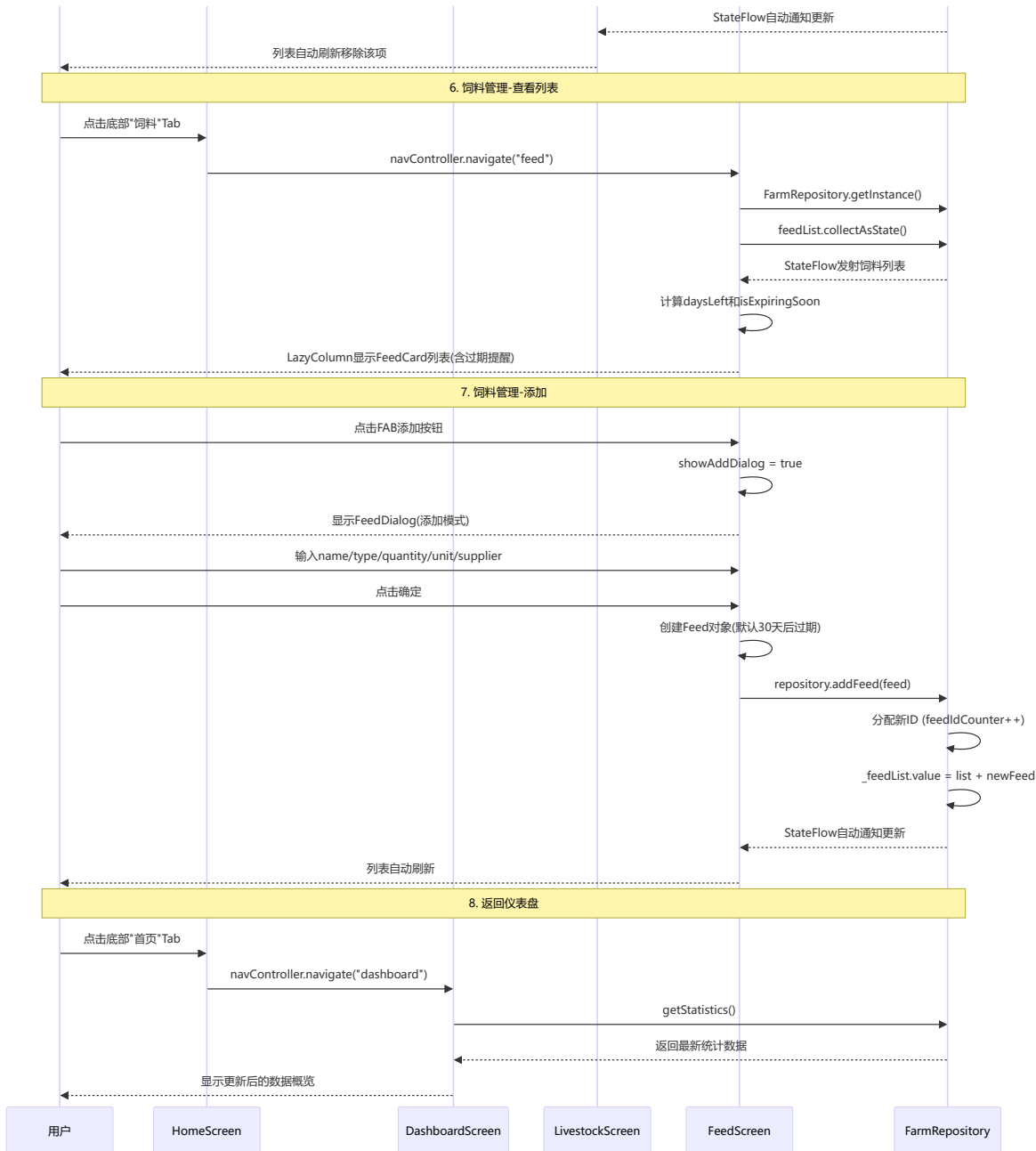


类职责说明：

类名	职责	主要方法/属性
FarmRepository	数据仓库，单例模式，管理所有数据	addLivestock、updateLivestock、deleteLivestock、addFeed、updateFeed、deleteFeed、getStatistics
Livestock	牲畜数据模型，data class	id、name、species、breed、age、weight、healthStatus、location、createTime
Feed	饲料数据模型，data class	id、name、type、quantity、unit、supplier、purchaseDate、expiryDate、createTime
FarmStatistics	统计数据模型，data class	totalLivestock、totalFeedTypes、totalFeedQuantity、speciesDistribution
DashboardScreen	仪表盘界面，展示数据概览	WelcomeCard、StatisticsCard、SpeciesDistributionCard、QuickActionsCard
LivestockScreen	牲畜管理界面，CRUD操作	LivestockCard、LivestockDialog、collectAsState监听数据
FeedScreen	饲料管理界面，含过期提醒	FeedCard、FeedDialog、过期天数计算

8.3 核心功能顺序图





顺序图说明：

该顺序图展示了系统的核心功能流程，包括8个主要步骤：

步骤	功能	关键操作
1	进入主页显示仪表盘	获取统计数据，显示欢迎卡片、数据概览、牲畜分布、快捷操作
2	牲畜管理-查看列表	通过collectAsState监听数据，LazyColumn显示列表
3	牲畜管理-添加	显示对话框，创建对象，分配ID，更新StateFlow
4	牲畜管理-编辑	预填数据，修改后更新，通过map操作替换
5	牲畜管理-删除	通过filter操作移除，StateFlow自动通知
6	饲料管理-查看列表	计算过期天数，显示过期提醒
7	饲料管理-添加	创建对象（默认30天后过期），分配ID
8	返回仪表盘	重新获取统计数据，显示最新信息

8.4 数据更新机制

响应式数据流：

用户操作 → UI组件 → Repository方法 → MutableStateFlow更新 → StateFlow发射 → collectAsState收集 → Compose重组 → UI自动刷新

关键技术点：

技术点	说明	代码示例
MutableStateFlow	可变状态流, 数据源	<pre>private val _livestockList = MutableStateFlow<List<Livestock>>(emptyList())</pre>
StateFlow	只读状态流, 对外暴露	<pre>val livestockList: StateFlow<List<Livestock>> = _livestockList.asStateFlow()</pre>
collectAsState()	Compose中收集 StateFlow	<pre>val livestockList by repository.livestockList.collectAsState()</pre>
remember	Compose状态记忆	<pre>val repository = remember { FarmRepository.getInstance() }</pre>
单例模式	Repository全局唯一	<pre>companion object { fun getInstance(): FarmRepository }</pre>

数据更新方式：

操作	Repository方法	StateFlow更新方式
添加牲畜	addLivestock()	<pre>_livestockList.value = _livestockList.value + newLivestock</pre>
编辑牲畜	updateLivestock()	<pre>_livestockList.value = _livestockList.value.map { }</pre>
删除牲畜	deleteLivestock()	<pre>_livestockList.value = _livestockList.value.filter { }</pre>
添加饲料	addFeed()	<pre>_feedList.value = _feedList.value + newFeed</pre>
编辑饲料	updateFeed()	<pre>_feedList.value = _feedList.value.map { }</pre>
删除饲料	deleteFeed()	<pre>_feedList.value = _feedList.value.filter { }</pre>

九、实验总结

9.1 完成情况

- ☑ Android登录页面功能完整
- ☑ Activity跳转与数据传递正常
- ☑ 输入验证逻辑正确
- ☑ 牲畜管理功能完整
- ☑ 饲料管理功能完整
- ☑ 数据统计仪表盘完成

9.2 心得体会

通过本次实验，我掌握了：

- 1. Jetpack Compose现代UI框架的使用方法
- 2. MVVM架构的设计思想和实现方式
- 3. Navigation组件的页面导航和数据传递
- 4. StateFlow响应式数据流的应用
- 5. Material Design 3组件的使用

9.3 遇到的问题及解决方案

问题	解决方案
Material Icons部分图标不可用	添加material-icons-extended依赖
启动Activity配置错误	修改AndroidManifest.xml中的启动Activity
StateFlow数据绑定问题	使用collectAsState()进行数据收集

十、参考资料

- 1. Android官方文档 - Jetpack Compose
- 2. Material Design 3 设计规范
- 3. Kotlin语言官方文档
- 4. Android Navigation组件使用指南